Products Public Two-Tier UX (PR 2 of 3) — Implementation Plan

Quick View dialog, technical detail page, per-product SEO + sitemap, order-form handoff — on the 0003 schema

Date: 2026-07-05
Status: Implementation plan (PR 2 of 3)
Owner: Fatima Abdul Raheem (CEO)
Branch: `feat/products-public`

Contents

1 Goal

For agentic workers: REQUIRED SUB-SKILL — use `superpowers:subagent-driven-development` (recommended) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

The two-tier public products experience — approachable Quick View dialog on the grid (with `?p=` deep links), a technical detail page with feature showcase + related posts + CTA rules, per-product SEO + sitemap, and an order-form handoff — all on the 0003 schema that PR 1 shipped.

2 Architecture

Server Components fetch; one client island (`ProductsExplorer`) owns filter pills + card grid + the Quick View (a native `<dialog>` — free focus trap + Esc). URL sync is `history.pushState` only (no RSC refetch). The detail page renders through a **pure, prop-driven** `ProductDetailView` so PR 3's preview can reuse it. "Markdown" fields render as blank-line-separated paragraphs — the established blog convention; a real renderer arrives with the blog admin (keeps the spec's "@dnd-kit is the one new dependency" promise: **zero new deps in this PR**).

Tech Stack: Next.js 16 App Router, React 19, Tailwind v4, Supabase (anon reads), Vitest + Testing Library.

3 Global Constraints

- Brand: **NurvexThink** / `nurvexthink`. Conventional Commits. Branch: `feat/products-public` (from current `main`).
- **No new npm dependencies.**
- Server Components by default; `"use client"` only at leaves (CLAUDE.md).
- Public reads via the anon client only; RLS is the security boundary (drafts 404 by absence).
- Spec contract: `docs/superpowers/specs/2026-07-04-products-experience-design.md` §2 (Public UX, CTA rules), §8.2.
- CTA rules (verbatim from spec): `live_url` set **and** `lifecycle != 'soon'` → primary **Open live app**; `lifecycle = 'soon'` or `live_url` empty → disabled **Coming soon**; `repo_url` set → secondary **View repo** (technical page only). Seed data uses `live_url = '#'` — treat `'#'` as empty.
- Quick View URL mechanics (verbatim): open writes `?p=<slug>` via `history.pushState`; state hydrates from `useSearchParams`; Back/Esc/backdrop close; unknown/draft/malformed `?p=` silently ignored; `/products` keeps `rel="canonical"` to itself; every published product in the sitemap.
- Definition of green: `npm run typecheck && npm run lint && npm test && npm run build` all exit 0.

4 File Structure

- `src/lib/content.ts` — **modify**: extend `Product`; add `ProductFeature`, `RelatedPost`, `ProductDetail` types.
- `src/lib/queries.ts` — **modify**: `toProduct` maps new fields; add `getProductDetailBySlug` (+ exported `toProductDetail`); keep `getProductBySlug` (order page uses it).
- `src/lib/queries.test.ts` — **modify**: cover new mapping (feature sort, draft-link filtering, CTA-relevant fields).

- `src/lib/product-cta.ts` — **create**: shared CTA-state helper + unit test file `src/lib/product-cta.test.t`
- `src/components/product-card.tsx` — **modify**: cover image, tagline, lifecycle/featured badges, `<a>` + `onOpen` interception.
- `src/components/products-explorer.tsx` — **create** (`"use client"`): pills + grid + Quick View state + URL sync.
- `src/components/product-quick-view.tsx` — **create** (`"use client"`): the native-`<dialog>` tier-1 box.
- `src/components/product-detail-view.tsx` — **create** (pure/prop-driven): tier-2 page body.
- `src/app/products/page.tsx` — **modify**: canonical, Suspense-wrapped explorer.
- `src/app/products/[slug]/page.tsx` — **modify**: fetch detail, render `ProductDetailView`, full `generateMetadata`.
- `src/app/order/page.tsx` + `src/components/order-form.tsx` — **modify**: `?ref=<slug>` pre-fills details.
- `src/app/sitemap.ts` — **create**: static routes + published products + posts.
- `supabase/migrations/0005_sample_features_and_links.sql` — **create**: demo features (FluxBoard, Pulse) + product ↔ post links so the new sections are visible.

5 Task 1: Data layer — view-model, detail query, CTA helper (TDD)

Files

- Modify: `src/lib/content.ts:89-100` (the `Product` type block)
- Modify: `src/lib/queries.ts` (imports, `toProduct`, add detail query; `toPost`/blog fns untouched)
- Modify: `src/lib/queries.test.ts`
- Create: `src/lib/product-cta.ts`, `src/lib/product-cta.test.ts`

Interfaces

- Consumes: `ProductRow`, `ProductFeatureRow` from `src/lib/supabase/types.ts` (PR 1).
- Produces (later tasks rely on exact names): types `Product { slug; name; category; tagline; summary; description; status: "Live"|"Beta"|"Soon"; tags: string[]; year; liveUrl; repoUrl: string | null; coverImage: string | null; highlights: string[]; featured: boolean }`, `ProductFeature { title; description; image }`, `RelatedPost { slug; title; excerpt; coverImage }`, `ProductDetail extends Product { descriptionParagraphs: string[]; technicalParagraphs: string[]; gallery: string[]; features: ProductFeature[]; relatedPosts: RelatedPost[]; seoDescription: string; ogImage: string | null }`; functions `toProduct(row)`, `toProductDetail(row)`, `getProductDetailBySlug(slug)`, `productCta(p: Pick<Product,"status"|"liveUrl">): { kind: "live"; href: string } | { kind: "soon" }`.

Step 1: Write failing tests

Replace `src/lib/queries.test.ts` content with:

```
import { describe, it, expect } from "vitest";
import { toProduct, toProductDetail } from "@/lib/queries";
import type { ProductRow } from "@/lib/supabase/types";

const row: ProductRow & {
```

```
  product_categories?: { name: string } | null;
  product_features?: {
    id: string; title: string; description: string | null;
    image: string | null; sort_order: number;
  }[];
  product_blog_links?: {
    sort_order: number;
    blog_posts: { slug: string; title: string; excerpt: string | null; cover_image:
        string | null } | null;
  }[];
} = {
  id: "00000000-0000-0000-0000-000000000001",
  slug: "fluxboard",
  name: "FluxBoard",
  tagline: "Standups into shipped work.",
  summary: "A keyboard-first project board.",
  highlights: ["Fast", "Realtime", "Keyboard-first"],
  description: "First para.\n\nSecond para.",
  technical_details: "Arch para.",
  cover_image: "https://cdn.example/cover.png",
  gallery: [],
  category_id: null,
  tech: ["Next.js"],
  lifecycle: "live",
  year: "2026",
  live_url: "https://fluxboard.app",
  repo_url: null,
  featured: true,
  sort_order: 10,
  status: "published",
  seo_description: null,
  og_image: null,
  published_at: "2026-07-01T00:00:00Z",
  created_at: "2026-06-01T00:00:00Z",
  updated_at: "2026-07-01T00:00:00Z",
  product_categories: { name: "Productivity" },
};

describe("toProduct", () => {
  it("maps lifecycle, tech, and the quick-view fields", () => {
    const p = toProduct(row);
    expect(p.status).toBe("Live");
    expect(p.tags).toEqual(["Next.js"]);
    expect(p.tagline).toBe("Standups into shipped work.");
    expect(p.highlights).toEqual(["Fast", "Realtime", "Keyboard-first"]);
    expect(p.coverImage).toBe("https://cdn.example/cover.png");
    expect(p.featured).toBe(true);
    expect(p.repoUrl).toBeNull();
  });

  it("falls back to summary when tagline is empty, and Software when category missing",
      () => {
    const p = toProduct({ ...row, tagline: null, product_categories: null });
    expect(p.tagline).toBe("A keyboard-first project board.");
    expect(p.category).toBe("Software");
  });
});

describe("toProductDetail", () => {
  it("splits description/technical into paragraphs", () => {
```

```
    const d = toProductDetail(row);
    expect(d.descriptionParagraphs).toEqual(["First para.", "Second para."]);
    expect(d.technicalParagraphs).toEqual(["Arch para."]);
  });

  it("orders features by sort_order and drops nothing", () => {
    const d = toProductDetail({
      ...row,
      product_features: [
        { id: "b", title: "B", description: null, image: null, sort_order: 20 },
        { id: "a", title: "A", description: "da", image: "ia", sort_order: 10 },
      ],
    });
    expect(d.features.map((f) => f.title)).toEqual(["A", "B"]);
  });

  it("orders related posts and filters draft posts (RLS returns them as null)", () => {
    const d = toProductDetail({
      ...row,
      product_blog_links: [
        { sort_order: 20, blog_posts: { slug: "two", title: "Two", excerpt: null,
            cover_image: null } },
        { sort_order: 10, blog_posts: { slug: "one", title: "One", excerpt: "e",
            cover_image: null } },
        { sort_order: 5, blog_posts: null },
      ],
    });
    expect(d.relatedPosts.map((p) => p.slug)).toEqual(["one", "two"]);
  });

  it("builds seoDescription fallback from tagline + summary", () => {
    const d = toProductDetail({ ...row, seo_description: null });
    expect(d.seoDescription).toBe(
      "Standups into shipped work. --A keyboard-first project board.",
    );
    expect(toProductDetail({ ...row, seo_description: "Custom." }).seoDescription).toBe("
        Custom.");
  });
});
```

Create `src/lib/product-cta.test.ts`:

```
import { describe, it, expect } from "vitest";
import { productCta } from "@/lib/product-cta";

describe("productCta", () => {
  it("returns live for a real URL on a live/beta product", () => {
    expect(productCta({ status: "Live", liveUrl: "https://x.app" })).toEqual({
      kind: "live",
      href: "https://x.app",
    });
    expect(productCta({ status: "Beta", liveUrl: "https://x.app" })).toEqual({
      kind: "live",
      href: "https://x.app",
    });
  });

  it("returns soon when lifecycle is Soon, regardless of URL", () => {
    expect(productCta({ status: "Soon", liveUrl: "https://x.app" })).toEqual({ kind: "
        soon" });
```

```
  });

  it("returns soon when the URL is empty or the '#' placeholder", () => {
    expect(productCta({ status: "Live", liveUrl: "#" })).toEqual({ kind: "soon" });
    expect(productCta({ status: "Live", liveUrl: "" })).toEqual({ kind: "soon" });
  });
});
```

Step 2: Run to verify failure

Run: `npx vitest run src/lib/queries.test.ts src/lib/product-cta.test.ts` — expect FAIL (`toProductDetail`, `product-cta` module missing; `tagline` not on `Product`).

Step 3: Extend the view-model

In `src/lib/content.ts`, replace the `Product` type block (lines 89–100) with:

```
// Products live in the `products` table; fetched via getProducts/getProductDetailBySlug
    .
export type Product = {
  slug: string;
  name: string;
  category: string;
  tagline: string;
  summary: string;
  description: string;
  status: "Live" | "Beta" | "Soon";
  tags: string[];
  year: string;
  liveUrl: string;
  repoUrl: string | null;
  coverImage: string | null;
  highlights: string[];
  featured: boolean;
};

export type ProductFeature = { title: string; description: string; image: string | null
    };

export type RelatedPost = { slug: string; title: string; excerpt: string; coverImage:
    string | null };

export type ProductDetail = Product & {
  descriptionParagraphs: string[];
  technicalParagraphs: string[];
  gallery: string[];
  features: ProductFeature[];
  relatedPosts: RelatedPost[];
  seoDescription: string;
  ogImage: string | null;
};
```

Step 4: Create `src/lib/product-cta.ts`

```
import type { Product } from "@/lib/content";

export type ProductCta = { kind: "live"; href: string } | { kind: "soon" };
```

```
/** Spec §2 CTA rules. The seed's placeholder '#' counts as no URL. */
export function productCta(p: Pick<Product, "status" | "liveUrl">): ProductCta {
  const hasUrl = Boolean(p.liveUrl) && p.liveUrl !== "#";
  if (p.status === "Soon" || !hasUrl) return { kind: "soon" };
  return { kind: "live", href: p.liveUrl };
}
```

Step 5: Update `src/lib/queries.ts`

Replace the product section (imports through `getProductBySlug`; blog fns untouched) with:

```
import { createPublicSupabaseClient } from "@/lib/supabase/public";
import type { Product, ProductDetail, BlogPost } from "@/lib/content";
import type { ProductRow, BlogPostRow } from "@/lib/supabase/types";

const LIFECYCLE_LABEL = { live: "Live", beta: "Beta", soon: "Soon" } as const;

/** Grid/detail selects. Nested selects are RLS-filtered per table for anon. */
const PRODUCT_SELECT = "*, product_categories(name)";
const PRODUCT_DETAIL_SELECT =
  "*, product_categories(name), product_features(id,title,description,image,sort_order),
      product_blog_links(sort_order, blog_posts(slug,title,excerpt,cover_image))";

type FeatureJoin = {
  id: string;
  title: string;
  description: string | null;
  image: string | null;
  sort_order: number;
};

type LinkJoin = {
  sort_order: number;
  blog_posts: {
    slug: string;
    title: string;
    excerpt: string | null;
    cover_image: string | null;
  } | null;
};

type ProductRowJoined = ProductRow & {
  product_categories?: { name: string } | null;
  product_features?: FeatureJoin[];
  product_blog_links?: LinkJoin[];
};

function paragraphs(text: string | null): string[] {
  return (text ?? "")
    .split(/\n\s*\n/)
    .map((p) => p.trim())
    .filter(Boolean);
}

export function toProduct(row: ProductRowJoined): Product {
  return {
    slug: row.slug,
    name: row.name,
    category: row.product_categories?.name ?? "Software",
```

```
    tagline: row.tagline?.trim() || (row.summary ?? ""),
    summary: row.summary ?? "",
    description: row.description ?? "",
    status: LIFECYCLE_LABEL[row.lifecycle] ?? "Live",
    tags: row.tech ?? [],
    year: row.year ?? "",
    liveUrl: row.live_url ?? "#",
    repoUrl: row.repo_url,
    coverImage: row.cover_image,
    highlights: row.highlights ?? [],
    featured: row.featured,
  };
}

export function toProductDetail(row: ProductRowJoined): ProductDetail {
  const base = toProduct(row);
  const features = [...(row.product_features ?? [])]
    .sort((a, b) => a.sort_order - b.sort_order)
    .map((f) => ({ title: f.title, description: f.description ?? "", image: f.image }));
  const relatedPosts = [...(row.product_blog_links ?? [])]
    .sort((a, b) => a.sort_order - b.sort_order)
    .flatMap((l) =>
      l.blog_posts
        ? [{
            slug: l.blog_posts.slug,
            title: l.blog_posts.title,
            excerpt: l.blog_posts.excerpt ?? "",
            coverImage: l.blog_posts.cover_image,
          }]
        : [],
    );
  return {
    ...base,
    descriptionParagraphs: paragraphs(row.description),
    technicalParagraphs: paragraphs(row.technical_details),
    gallery: row.gallery ?? [],
    features,
    relatedPosts,
    seoDescription:
      row.seo_description?.trim() ||
      [base.tagline, base.summary].filter(Boolean).join(" --"),
    ogImage: row.og_image ?? row.cover_image,
  };
}

export async function getProducts(): Promise<Product[]> {
  try {
    const supabase = createPublicSupabaseClient();
    const { data, error } = await supabase
      .from("products")
      .select(PRODUCT_SELECT)
      .eq("status", "published")
      .order("featured", { ascending: false })
      .order("sort_order", { ascending: true })
      .order("created_at", { ascending: false });
    if (error || !data) return [];
    return data.map(toProduct);
  } catch {
    return [];
  }
```

```
}

export async function getProductBySlug(slug: string): Promise<Product | null> {
  try {
    const supabase = createPublicSupabaseClient();
    const { data, error } = await supabase
      .from("products")
      .select(PRODUCT_SELECT)
      .eq("slug", slug)
      .eq("status", "published")
      .maybeSingle();
    if (error || !data) return null;
    return toProduct(data);
  } catch {
    return null;
  }
}

export async function getProductDetailBySlug(slug: string): Promise<ProductDetail | null
    > {
  try {
    const supabase = createPublicSupabaseClient();
    const { data, error } = await supabase
      .from("products")
      .select(PRODUCT_DETAIL_SELECT)
      .eq("slug", slug)
      .eq("status", "published")
      .maybeSingle();
    if (error || !data) return null;
    return toProductDetail(data);
  } catch {
    return null;
  }
}
```

Note on the seoDescription test expectation: with `tagline = "Standups into shipped work."` and summary present, the join produces `"Standups into shipped work.  -- A keyboard-first project board."`

Step 6: Run tests — pass

Run: `npx vitest run src/lib/queries.test.ts src/lib/product-cta.test.ts` → all green (9 tests).

Step 7: Typecheck

Run: `npm run typecheck` — expect **failures only in** `product-card.tsx` / **detail page are NOT expected**: the extended `Product` adds fields (additive), so typecheck should pass. If `STATUS_STYLES` or other consumers error, stop and report (don't fix ahead of Task 2).

Step 8: Commit

```
git add src/lib/content.ts src/lib/queries.ts src/lib/queries.test.ts src/lib/product-
    cta.ts src/lib/product-cta.test.ts
git commit -m "feat: product detail data layer --quick-view fields, features, related
    posts, CTA rules"
```

6 Task 2: Grid, cards, Quick View dialog with URL sync

Files

- Modify: `src/components/product-card.tsx` (whole-file replace below)
- Create: `src/components/product-quick-view.tsx`
- Create: `src/components/products-explorer.tsx`
- Create: `src/components/products-explorer.test.tsx`
- Modify: `src/app/products/page.tsx`

Interfaces

- Consumes: `Product`, `productCta` from Task 1.
- Produces: `<ProductsExplorer products={Product[]} />` (client, must sit under `<Suspense>`); `<ProductCard product onOpen?>`; `<ProductQuickView product open onClose />`. PR 3's preview will reuse `ProductQuickView` with a static product.

Step 1: Replace `src/components/product-card.tsx`

```
import Image from "next/image";
import { Sparkles } from "lucide-react";
import type { Product } from "@/lib/content";
import { cn } from "@/lib/utils";

const STATUS_STYLES: Record<Product["status"], string> = {
  Live: "bg-emerald-500/12 text-emerald-500 ring-emerald-500/20",
  Beta: "bg-amber-500/12 text-amber-500 ring-amber-500/20",
  Soon: "bg-muted text-muted-foreground ring-border",
};

/**
 * A real link to the technical page; when `onOpen` is provided the click is
 * intercepted to open the Quick View instead (crawlers/no-JS still navigate).
 */
export function ProductCard({
  product,
  onOpen,
}: {
  product: Product;
  onOpen?: (slug: string) => void;
}) {
  return (
    <a
      href={`/products/${product.slug}`}
      onClick={
        onOpen
          ? (e) => {
              if (e.metaKey || e.ctrlKey || e.shiftKey || e.altKey) return;
              e.preventDefault();
              onOpen(product.slug);
            }
          : undefined
      }
      className="group border-border bg-card hover:border-primary/40 relative flex flex-
          col overflow-hidden rounded-2xl border transition-colors"
    >
```

```
      <div className="bg-muted relative aspect-[16/9] w-full overflow-hidden">
        {product.coverImage ? (
          <Image
            src={product.coverImage}
            alt=""
            fill
            sizes="(min-width: 1024px) 30vw, (min-width: 640px) 45vw, 90vw"
            className="object-cover transition-transform duration-500 group-hover:scale
                -[1.03]"
          />
        ) : (
          <div
            aria-hidden
            className="from-brand-navy/40 to-brand-indigo/15 absolute inset-0 bg-gradient-
                to-br"
          />
        )}
        {product.featured ? (
          <span className="bg-background/80 text-primary absolute top-3 left-3 inline-
              flex items-center gap-1 rounded-full px-2.5 py-0.5 text-xs font-medium
              backdrop-blur">
            <Sparkles className="size-3" />
            Featured
          </span>
        ) : null}
      </div>

      <div className="flex flex-1 flex-col gap-3 p-6">
        <div className="flex items-center justify-between">
          <span className="text-muted-foreground font-mono text-xs tracking-[0.18em]
              uppercase">
            {product.category}
          </span>
          <span
            className={cn(
              "rounded-full px-2.5 py-0.5 text-xs font-medium ring-1 ring-inset",
              STATUS_STYLES[product.status],
            )}
          >
            {product.status}
          </span>
        </div>
        <div className="flex flex-col gap-1.5">
          <h3 className="font-heading text-xl font-semibold tracking-tight">{product.name
              }</h3>
          <p className="text-muted-foreground text-sm">{product.tagline}</p>
        </div>
      </div>
    </a>
  );
}
```

Step 2: Create `src/components/product-quick-view.tsx`

```
"use client";

import { useEffect, useRef } from "react";
import Image from "next/image";
```

```
import Link from "next/link";
import { ArrowRight, ArrowUpRight, Check } from "lucide-react";
import type { Product } from "@/lib/content";
import { productCta } from "@/lib/product-cta";
import { buttonVariants } from "@/components/ui/button";
import { cn } from "@/lib/utils";

/**
 * Tier 1: the approachable box. A native <dialog> --free focus trap + Esc.
 * The parent owns open state and URL sync; Esc surfaces as the dialog's
 * 'close' event, backdrop clicks hit the <dialog> element itself.
 */
export function ProductQuickView({
  product,
  open,
  onClose,
}: {
  product: Product | null;
  open: boolean;
  onClose: () => void;
}) {
  const ref = useRef<HTMLDialogElement>(null);

  useEffect(() => {
    const dialog = ref.current;
    if (!dialog) return;
    if (open && !dialog.open) dialog.showModal();
    if (!open && dialog.open) dialog.close();
  }, [open]);

  if (!product) return null;
  const cta = productCta(product);

  return (
    <dialog
      ref={ref}
      aria-label={product.name}
      onClose={onClose}
      onClick={(e) => {
        if (e.target === ref.current) onClose(); // backdrop
      }}
      className="bg-card text-foreground border-border m-auto w-[min(92vw,34rem)] rounded
          -2xl border p-0 shadow-2xl backdrop:bg-black/60 backdrop:backdrop-blur-sm"
    >
      <div className="flex flex-col">
        <div className="bg-muted relative aspect-[16/9] w-full overflow-hidden rounded-t
            -2xl">
          {product.coverImage ? (
            <Image src={product.coverImage} alt="" fill sizes="34rem" className="object-
                cover" />
          ) : (
            <div
              aria-hidden
              className="from-brand-navy/40 to-brand-indigo/15 absolute inset-0 bg-
                  gradient-to-br"
            />
          )}
        </div>

        <div className="flex flex-col gap-4 p-6">
```

```
          <div className="text-muted-foreground flex items-center gap-3 font-mono text-xs
               tracking-[0.18em] uppercase">
            <span className="text-primary">{product.category}</span>
            <span aria-hidden>·</span>
            <span>{product.status}</span>
          </div>
          <div className="flex flex-col gap-1.5">
            <h2 className="font-heading text-2xl font-bold tracking-tight">{product.name
                }</h2>
            <p className="text-muted-foreground text-pretty">{product.tagline}</p>
          </div>

          {product.highlights.length > 0 ? (
            <ul className="flex flex-col gap-2">
              {product.highlights.slice(0, 6).map((h) => (
                <li key={h} className="flex items-start gap-2.5 text-sm">
                  <span className="bg-primary/10 text-primary mt-0.5 inline-flex size-5
                      shrink-0 items-center justify-center rounded-full">
                    <Check className="size-3" />
                  </span>
                  {h}
                </li>
              ))}
            </ul>
          ) : (
            <p className="text-muted-foreground text-sm">{product.summary}</p>
          )}

          <div className="flex flex-col gap-2.5 pt-1 sm:flex-row">
            {cta.kind === "live" ? (
              <a
                href={cta.href}
                target="_blank"
                rel="noopener noreferrer"
                className={cn(buttonVariants(), "group")}
              >
                Open live app
                <ArrowUpRight className="size-4 transition-transform group-hover:translate
                    -x-0.5 group-hover:-translate-y-0.5" />
              </a>
            ) : (
              <span className={cn(buttonVariants(), "pointer-events-none opacity-60")}>
                Coming soon
              </span>
            )}
            <Link
              href={`/products/${product.slug}`}
              className={cn(buttonVariants({ variant: "outline" }), "group")}
            >
              Full technical details
              <ArrowRight className="size-4 transition-transform group-hover:translate-x
                  -0.5" />
            </Link>
          </div>
        </div>
      </div>
    </dialog>
  );
}
```

Step 3: Create `src/components/products-explorer.tsx`

```
"use client";

import { useCallback, useEffect, useMemo, useState } from "react";
import { useSearchParams } from "next/navigation";
import type { Product } from "@/lib/content";
import { ProductCard } from "@/components/product-card";
import { ProductQuickView } from "@/components/product-quick-view";
import { cn } from "@/lib/utils";

/**
 * Client island for /products: category pills + grid + Quick View.
 * URL sync is pushState-only (spec §2): no router.push, no RSC refetch.
 * Initial open state hydrates from ?p=; unknown slugs are silently ignored.
 */
export function ProductsExplorer({ products }: { products: Product[] }) {
  const searchParams = useSearchParams();
  const [openSlug, setOpenSlug] = useState<string | null>(null);
  const [filter, setFilter] = useState<string>("All");

  const bySlug = useMemo(() => new Map(products.map((p) => [p.slug, p])), [products]);

  // Hydrate from ?p= and follow browser navigation (Back/Forward).
  useEffect(() => {
    const fromUrl = searchParams.get("p");
    setOpenSlug(fromUrl && bySlug.has(fromUrl) ? fromUrl : null);
  }, [searchParams, bySlug]);

  useEffect(() => {
    const onPop = () => {
      const p = new URLSearchParams(window.location.search).get("p");
      setOpenSlug(p && bySlug.has(p) ? p : null);
    };
    window.addEventListener("popstate", onPop);
    return () => window.removeEventListener("popstate", onPop);
  }, [bySlug]);

  const openQuickView = useCallback((slug: string) => {
    const url = new URL(window.location.href);
    url.searchParams.set("p", slug);
    window.history.pushState(null, "", url);
    setOpenSlug(slug);
  }, []);

  const closeQuickView = useCallback(() => {
    setOpenSlug(null);
    const url = new URL(window.location.href);
    if (url.searchParams.has("p")) {
      url.searchParams.delete("p");
      window.history.pushState(null, "", url);
    }
  }, []);

  const categories = useMemo(() => {
    const names = Array.from(new Set(products.map((p) => p.category)));
    return ["All", ...names];
  }, [products]);

  const visible =
```

```
    filter === "All" ? products : products.filter((p) => p.category === filter);

  return (
    <div className="flex flex-col gap-8">
      {categories.length > 2 ? (
        <div className="flex flex-wrap gap-2" role="group" aria-label="Filter by category
            ">
          {categories.map((c) => (
            <button
              key={c}
              type="button"
              onClick={() => setFilter(c)}
              aria-pressed={filter === c}
              className={cn(
                "rounded-full border px-3.5 py-1.5 text-sm transition-colors",
                filter === c
                  ? "border-primary bg-primary/10 text-primary"
                  : "border-border text-muted-foreground hover:text-foreground",
              )}
            >
              {c}
            </button>
          ))}
        </div>
      ) : null}

      {visible.length === 0 ? (
        <p className="text-muted-foreground">Nothing in this category yet.</p>
      ) : (
        <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
          {visible.map((product) => (
            <ProductCard key={product.slug} product={product} onOpen={openQuickView} />
          ))}
        </div>
      )}

      <ProductQuickView
        product={openSlug ? (bySlug.get(openSlug) ?? null) : null}
        open={openSlug !== null}
        onClose={closeQuickView}
      />
    </div>
  );
}
```

Step 4: Update `src/app/products/page.tsx`

Replace the file with:

```
import type { Metadata } from "next";
import { Suspense } from "react";
import { Container } from "@/components/ui/container";
import { Eyebrow } from "@/components/section-heading";
import { ProductsExplorer } from "@/components/products-explorer";
import { getProducts } from "@/lib/queries";

export const dynamic = "force-dynamic";

export const metadata: Metadata = {
```

```
  title: "Products",
  description: "Software NurvexThink designs, builds, and ships --explore the catalog.",
  alternates: { canonical: "/products" },
};

export default async function ProductsPage() {
  const products = await getProducts();

  return (
    <>
      <section className="border-border relative overflow-hidden border-b">
        <div aria-hidden className="bg-grid mask-fade-y absolute inset-0" />
        <Container className="relative py-20 sm:py-24">
          <div className="flex max-w-2xl flex-col gap-5">
            <Eyebrow>Catalog</Eyebrow>
            <h1 className="font-heading text-4xl font-bold tracking-tight sm:text-5xl">
                Products</h1>
            <p className="text-muted-foreground text-lg text-pretty">
              Software we build, run, and keep improving. Click any product for the short
                  story --
              or go straight to the technical details.
            </p>
          </div>
        </Container>
      </section>

      <section className="py-16 sm:py-20">
        <Container>
          {products.length === 0 ? (
            <p className="text-muted-foreground">No products published yet --check back
                soon.</p>
          ) : (
            <Suspense fallback={null}>
              <ProductsExplorer products={products} />
            </Suspense>
          )}
        </Container>
      </section>
    </>
  );
}
```

Step 5: Component test

Create `src/components/products-explorer.test.tsx`:

```
import { describe, it, expect, vi, beforeEach } from "vitest";
import { render, screen, fireEvent } from "@testing-library/react";
import { ProductsExplorer } from "@/components/products-explorer";
import type { Product } from "@/lib/content";

const params = vi.hoisted(() => ({ value: new URLSearchParams() }));
vi.mock("next/navigation", () => ({
  useSearchParams: () => params.value,
}));

function makeProduct(over: Partial<Product>): Product {
  return {
    slug: "fluxboard",
```

```
    name: "FluxBoard",
    category: "Productivity",
    tagline: "Standups into shipped work.",
    summary: "Sum",
    description: "",
    status: "Live",
    tags: [],
    year: "2026",
    liveUrl: "#",
    repoUrl: null,
    coverImage: null,
    highlights: ["One", "Two", "Three"],
    featured: true,
    ...over,
  };
}

const products = [
  makeProduct({}),
  makeProduct({ slug: "pulse", name: "Pulse", category: "Analytics", featured: false }),
];

// jsdom has no <dialog> implementation.
beforeEach(() => {
  params.value = new URLSearchParams();
  HTMLDialogElement.prototype.showModal = vi.fn(function (this: HTMLDialogElement) {
    this.setAttribute("open", "");
  });
  HTMLDialogElement.prototype.close = vi.fn(function (this: HTMLDialogElement) {
    this.removeAttribute("open");
  });
});

describe("ProductsExplorer", () => {
  it("cards are real links to the technical page", () => {
    render(<ProductsExplorer products={products} />);
    const link = screen.getAllByRole("link").find((a) =>
      a.getAttribute("href")?.endsWith("/products/fluxboard"),
    );
    expect(link).toBeTruthy();
  });

  it("clicking a card opens the quick view and pushes ?p=", () => {
    const push = vi.spyOn(window.history, "pushState");
    render(<ProductsExplorer products={products} />);
    fireEvent.click(screen.getByText("Standups into shipped work."));
    expect(screen.getByText("Full technical details")).toBeInTheDocument();
    expect(push).toHaveBeenCalled();
    expect(String(push.mock.calls[0][2])).toContain("p=fluxboard");
  });

  it("hydrates open state from ?p= and ignores unknown slugs", () => {
    params.value = new URLSearchParams("p=pulse");
    render(<ProductsExplorer products={products} />);
    expect(screen.getByText("Full technical details")).toBeInTheDocument();

    params.value = new URLSearchParams("p=not-a-product");
    render(<ProductsExplorer products={products} />);
  });
```

```
  it("filters by category pill", () => {
    render(<ProductsExplorer products={products} />);
    fireEvent.click(screen.getByRole("button", { name: "Analytics" }));
    expect(screen.queryByText("Standups into shipped work.")).not.toBeInTheDocument();
    expect(screen.getByText("Pulse")).toBeInTheDocument();
  });
});
```

Step 6: Run tests

Run: `npx vitest run src/components/products-explorer.test.tsx` → PASS.

Step 7: Green bar

Run: `npm run typecheck && npm run lint && npm test && npm run build` → all green.

Step 8: Commit

```
git add src/components/product-card.tsx src/components/product-quick-view.tsx src/
    components/products-explorer.tsx src/components/products-explorer.test.tsx src/app/
    products/page.tsx
git commit -m "feat: products grid with quick-view dialog --?p= deep links, category
    pills"
```

7 Task 3: Technical detail page (prop-driven) + order-form handoff

Files

- Create: `src/components/product-detail-view.tsx`
- Modify: `src/app/products/[slug]/page.tsx` (whole-file replace)
- Modify: `src/app/order/page.tsx`, `src/components/order-form.tsx`

Interfaces

- Consumes: `ProductDetail`, `getProductDetailBySlug`, `getProductBySlug`, `productCta` (Task 1).
- Produces: `<ProductDetailView detail={ProductDetail} />` — pure; PR 3's preview route will render it with draft data. `OrderForm` gains optional `defaultDetails?: string`.

Step 1: Create `src/components/product-detail-view.tsx`

(pure — no data fetching):

```
import Image from "next/image";
import Link from "next/link";
import { ArrowLeft, ArrowRight, ArrowUpRight, Check } from "lucide-react";
import type { ProductDetail } from "@/lib/content";
import { productCta } from "@/lib/product-cta";
import { Container } from "@/components/ui/container";
import { buttonVariants } from "@/components/ui/button";
import { cn } from "@/lib/utils";

/** Tier 2: the technical page body. Pure so PR 3's admin preview can reuse it. */
export function ProductDetailView({ detail }: { detail: ProductDetail }) {
  const cta = productCta(detail);
```

```
return (
  <section className="py-16 sm:py-24">
    <Container className="flex max-w-4xl flex-col gap-12">
      <div className="flex flex-col gap-8">
        <Link
          href="/products"
          className="text-muted-foreground hover:text-foreground inline-flex items-
              center gap-1.5 text-sm transition-colors"
        >
          <ArrowLeft className="size-4" />
          All products
        </Link>

        {/* Overview strip --same story as the Quick View. */}
        <div className="flex flex-col gap-4">
          <div className="text-muted-foreground flex items-center gap-3 font-mono text-
              xs tracking-[0.18em] uppercase">
            <span className="text-primary">{detail.category}</span>
            <span aria-hidden>·</span>
            <span>{detail.status}</span>
            {detail.year ? (
              <>
                <span aria-hidden>·</span>
                <span>{detail.year}</span>
              </>
            ) : null}
          </div>
          <h1 className="font-heading text-4xl font-bold tracking-tight sm:text-5xl">
            {detail.name}
          </h1>
          <p className="text-muted-foreground text-lg text-pretty">{detail.tagline}</p>
        </div>

        {detail.coverImage ? (
          <div className="bg-muted relative aspect-[16/9] overflow-hidden rounded-2xl">
            <Image
              src={detail.coverImage}
              alt={`${detail.name} cover`}
              fill
              priority
              sizes="(min-width: 1024px) 56rem, 92vw"
              className="object-cover"
            />
          </div>
        ) : null}

        {detail.highlights.length > 0 ? (
          <ul className="grid gap-2.5 sm:grid-cols-2">
            {detail.highlights.map((h) => (
              <li key={h} className="flex items-start gap-2.5 text-sm">
                <span className="bg-primary/10 text-primary mt-0.5 inline-flex size-5
                    shrink-0 items-center justify-center rounded-full">
                  <Check className="size-3" />
                </span>
                {h}
              </li>
            ))}
          </ul>
        ) : null}
```

```
          <div className="flex flex-col gap-3 sm:flex-row">
            {cta.kind === "live" ? (
              <a
                href={cta.href}
                target="_blank"
                rel="noopener noreferrer"
                className={cn(buttonVariants({ size: "lg" }), "group")}
              >
                Open live app
                <ArrowUpRight className="size-4 transition-transform group-hover:translate
                    -x-0.5 group-hover:-translate-y-0.5" />
              </a>
            ) : (
              <span className={cn(buttonVariants({ size: "lg" }), "pointer-events-none
                  opacity-60")}>
                Coming soon
              </span>
            )}
            {detail.repoUrl ? (
              <a
                href={detail.repoUrl}
                target="_blank"
                rel="noopener noreferrer"
                className={buttonVariants({ variant: "outline", size: "lg" })}
              >
                View repo
              </a>
            ) : null}
          </div>
        </div>

        {/* Feature showcase --alternating image/text rows; hidden when empty. */}
        {detail.features.length > 0 ? (
          <div className="flex flex-col gap-6">
            <h2 className="font-heading text-2xl font-bold tracking-tight">Features</h2>
            <div className="flex flex-col gap-6">
              {detail.features.map((feature, i) => (
                <div
                  key={feature.title}
                  className={cn(
                    "border-border bg-card grid gap-6 overflow-hidden rounded-2xl border
                        sm:grid-cols-2",
                    i % 2 === 1 && "sm:[&>*:first-child]:order-2",
                  )}
                >
                  <div className="bg-muted relative min-h-48">
                    {feature.image ? (
                      <Image
                        src={feature.image}
                        alt={`${feature.title} illustration`}
                        fill
                        sizes="(min-width: 640px) 28rem, 92vw"
                        className="object-cover"
                      />
                    ) : (
                      <div
                        aria-hidden
                        className="from-brand-navy/30 to-brand-indigo/10 absolute inset-0
                            bg-gradient-to-br"
```

```
                  />
                )}
              </div>
              <div className="flex flex-col justify-center gap-2 p-6 sm:p-8">
                <h3 className="font-heading text-lg font-semibold tracking-tight">
                  {feature.title}
                </h3>
                <p className="text-muted-foreground text-sm leading-relaxed">
                  {feature.description}
                </p>
              </div>
            </div>
          ))}
        </div>
      </div>
    ) : null}

    {/* Technical body. */}
    <div className="flex flex-col gap-6">
      {detail.descriptionParagraphs.length > 0 ? (
        <div className="border-border bg-card flex flex-col gap-4 rounded-2xl border p
            -6 sm:p-8">
          {detail.descriptionParagraphs.map((p) => (
            <p key={p.slice(0, 40)} className="text-foreground/90 text-base leading-
                relaxed">
              {p}
            </p>
          ))}
        </div>
      ) : null}

      {detail.technicalParagraphs.length > 0 ? (
        <div className="flex flex-col gap-4">
          <h2 className="font-heading text-2xl font-bold tracking-tight">Technical
              details</h2>
          <div className="border-border bg-card flex flex-col gap-4 rounded-2xl border
               p-6 sm:p-8">
            {detail.technicalParagraphs.map((p) => (
              <p key={p.slice(0, 40)} className="text-foreground/90 text-base leading-
                  relaxed">
                {p}
              </p>
            ))}
          </div>
        </div>
      ) : null}

      {detail.tags.length > 0 ? (
        <div className="flex flex-wrap gap-2">
          {detail.tags.map((tag) => (
            <span
              key={tag}
              className="border-border bg-muted text-muted-foreground rounded-md
                  border px-2.5 py-1 text-xs"
            >
              {tag}
            </span>
          ))}
        </div>
      ) : null}
```

```
        {detail.gallery.length > 0 ? (
          <div className="grid gap-4 sm:grid-cols-2">
            {detail.gallery.map((src) => (
              <div key={src} className="bg-muted relative aspect-[16/10] overflow-hidden
                  rounded-xl">
                <Image
                  src={src}
                  alt=""
                  fill
                  sizes="(min-width: 640px) 28rem, 92vw"
                  className="object-cover"
                />
              </div>
            ))}
          </div>
        ) : null}
      </div>

      {/* Related posts --hidden when none. */}
      {detail.relatedPosts.length > 0 ? (
        <div className="flex flex-col gap-6">
          <h2 className="font-heading text-2xl font-bold tracking-tight">Related writing
              </h2>
          <div className="grid gap-4 sm:grid-cols-2">
            {detail.relatedPosts.map((post) => (
              <Link
                key={post.slug}
                href={`/blog/${post.slug}`}
                className="group border-border bg-card hover:border-primary/40 flex flex
                    -col gap-2 rounded-2xl border p-6 transition-colors"
              >
                <h3 className="font-heading flex items-center gap-1.5 text-lg font-
                    semibold tracking-tight">
                  {post.title}
                  <ArrowRight className="text-muted-foreground group-hover:text-primary
                      size-4 transition-transform group-hover:translate-x-0.5" />
                </h3>
                <p className="text-muted-foreground line-clamp-2 text-sm">{post.excerpt
                    }</p>
              </Link>
            ))}
          </div>
        </div>
      ) : null}

      {/* Order handoff. */}
      <div className="border-border bg-card flex flex-col items-start justify-between
          gap-4 rounded-2xl border p-6 sm:flex-row sm:items-center sm:p-8">
        <div className="flex flex-col gap-1">
          <h2 className="font-heading text-xl font-semibold tracking-tight">
            Want something like {detail.name}?
          </h2>
          <p className="text-muted-foreground text-sm">
            We build custom software on demand --tell us what you need.
          </p>
        </div>
        <Link
          href={`/order?ref=${detail.slug}`}
          className={cn(buttonVariants({ size: "lg" }), "group shrink-0")}
```

```
          >
            Request something like this
            <ArrowRight className="size-4 transition-transform group-hover:translate-x
                -0.5" />
          </Link>
        </div>
      </Container>
    </section>
  );
}
```

Step 2: Replace `src/app/products/[slug]/page.tsx`

```
import type { Metadata } from "next";
import { notFound } from "next/navigation";
import { ProductDetailView } from "@/components/product-detail-view";
import { getProductDetailBySlug } from "@/lib/queries";

export const dynamic = "force-dynamic";

export async function generateMetadata({
  params,
}: {
  params: Promise<{ slug: string }>;
}): Promise<Metadata> {
  const { slug } = await params;
  const detail = await getProductDetailBySlug(slug);
  if (!detail) return { title: "Product not found" };
  return {
    title: detail.name,
    description: detail.seoDescription,
    alternates: { canonical: `/products/${detail.slug}` },
    openGraph: {
      title: `${detail.name} --NurvexThink`,
      description: detail.seoDescription,
      ...(detail.ogImage ? { images: [{ url: detail.ogImage }] } : {}),
    },
  };
}

export default async function ProductDetailPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;
  const detail = await getProductDetailBySlug(slug);
  if (!detail) notFound();
  return <ProductDetailView detail={detail} />;
}
```

Step 3: Order handoff

In `src/components/order-form.tsx`, add the prop and use it:

- Change the component signature `export function OrderForm()` to `export function OrderForm({ defaultDetails = "" }: { defaultDetails?: string })`.
- On the `<textarea name="details" ...>` add `defaultValue={defaultDetails}`.

In `src/app/order/page.tsx`:

- Make the page async with `searchParams`:

```
export default async function OrderPage({
  searchParams,
}: {
  searchParams: Promise<{ ref?: string }>;
}) {
  const { ref } = await searchParams;
  const refProduct = ref ? await getProductBySlug(ref) : null;
  const defaultDetails = refProduct
    ? `I'd like something like ${refProduct.name} --`
    : "";
  // ... existing JSX unchanged, except:
  // <OrderForm defaultDetails={defaultDetails} />
}
```

- Add the import: `import { getProductBySlug } from "@/lib/queries";`
- An unknown/missing `ref` silently falls back to the empty default (never an error).

Step 4: Green bar

Run: `npm run typecheck && npm run lint && npm test && npm run build`.

Step 5: Commit

```
git add src/components/product-detail-view.tsx "src/app/products/[slug]/page.tsx" src/
    app/order/page.tsx src/components/order-form.tsx
git commit -m "feat: two-tier technical page --feature showcase, related posts, CTA
    rules, order handoff"
```

8 Task 4: Sitemap + sample-content seed

Files

- Create: `src/app/sitemap.ts`
- Create: `supabase/migrations/0005_sample_features_and_links.sql`

Interfaces

- Consumes: `getProducts`, `getPosts` (existing).
- Produces: `/sitemap.xml` route; seed file the owner runs in the SQL editor.

Step 1: Create `src/app/sitemap.ts`

```
import type { MetadataRoute } from "next";
import { getProducts, getPosts } from "@/lib/queries";

const BASE = "https://nurvexthink-website.vercel.app";

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const [products, posts] = await Promise.all([getProducts(), getPosts()]);

  const staticRoutes = ["", "/products", "/blog", "/about", "/contact", "/order"].map(
```

```
    (path) => ({ url: `${BASE}${path}`, lastModified: new Date() }),
  );

  return [
    ...staticRoutes,
    ...products.map((p) => ({ url: `${BASE}/products/${p.slug}`, lastModified: new Date()
        })),
    ...posts.map((p) => ({ url: `${BASE}/blog/${p.slug}`, lastModified: new Date() })),
  ];
}
```

Step 2: Create supabase/migrations/0005_sample_features_and_links.sql

```
-- NurvexThink --0005: sample features + product↔blog links so the new
-- public sections are visible with real content. Idempotent.

-- FluxBoard features
insert into public.product_features (product_id, title, description, sort_order)
select p.id, f.title, f.description, f.sort_order
from public.products p,
     (values
       ($s$Keyboard-first flow$s$, $s$Every action has a shortcut --plan, assign, and
           move work without touching the mouse.$s$, 10),
       ($s$Realtime sync$s$, $s$Everyone sees the same board, instantly. No refresh
           button, no "who has the latest?"$s$, 20)
     ) as f(title, description, sort_order)
where p.slug = $s$fluxboard$s$
  and not exists (
    select 1 from public.product_features x
    where x.product_id = p.id and x.title = f.title
  );

-- Pulse features
insert into public.product_features (product_id, title, description, sort_order)
select p.id, f.title, f.description, f.sort_order
from public.products p,
     (values
       ($s$Funnels in seconds$s$, $s$Pick the events, see the drop-off. No SQL, no
           waiting on a data team.$s$, 10),
       ($s$Privacy-first$s$, $s$No cookies, no fingerprinting --analytics you don't need
           a banner for.$s$, 20)
     ) as f(title, description, sort_order)
where p.slug = $s$pulse$s$
  and not exists (
    select 1 from public.product_features x
    where x.product_id = p.id and x.title = f.title
  );

-- Product ↔blog links
insert into public.product_blog_links (product_id, blog_post_id, sort_order)
select p.id, b.id, l.sort_order
from (values
       ($s$fluxboard$s$, $s$a-preview-link-from-day-one$s$, 10),
       ($s$fluxboard$s$, $s$why-we-publish-our-own-software$s$, 20),
       ($s$pulse$s$, $s$scaling-to-zero-with-supabase$s$, 10)
     ) as l(product_slug, post_slug, sort_order)
join public.products p on p.slug = l.product_slug
join public.blog_posts b on b.slug = l.post_slug
```

```
on conflict (product_id, blog_post_id) do update set sort_order = excluded.sort_order;

select
  (select count(*) from public.product_features) as features,
  (select count(*) from public.product_blog_links) as links;
-- Expect: features = 4, links = 3.
```

Step 3: Green bar

(build proves the sitemap route compiles): `npm run typecheck && npm run lint && npm test && npm run build`.

Step 4: Commit

```
git add src/app/sitemap.ts supabase/migrations/0005_sample_features_and_links.sql
git commit -m "feat: sitemap with product/blog URLs; sample features + related-post
    links seed"
```

9 Task 5: Visual QA, plan PDF twin, PR

(Controller-level task — screenshots need the local server and human-visible artifacts.)

Step 1

`npm run build` + start locally; screenshot `/products` (grid + open Quick View via `?p=fluxboard`) and `/products/fluxboard` in dark and light themes; visually check dialog focus, badges, CTA states ("Coming soon" on Vault, live on none yet since seeds use `#`).

Step 2

Generate `docs/superpowers/plans/2026-07-05-products-public.tex` + PDF (brand template, lstlistings), commit.

Step 3

Push `feat/products-public`, open the PR with screenshots; note that `0005_sample_features_and_links.sql` should be run in the SQL editor (any time — sections hide until it lands).

Step 4

After merge + deploy: verify Quick View, deep link `?p=pulse`, technical page sections, `sitemap.xml` on production.

10 Self-Review (done)

- **Spec §2 coverage:** grid card contents (done) (cover/tagline/category/lifecycle/ featured; tech chips moved to tier 2); pills from live categories with empty ones hidden (done) (derived from published products — same visible behavior); order (done) (query from PR 1); real-link cards with interception (done); Quick View content + URL mechanics + silent-ignore + canonical + sitemap (done); tier-2 overview strip, feature showcase (alternating, hidden-when-empty), technical body, gallery, related posts (published only via RLS null-filtering), order CTA (done); CTA rules incl. `'#'`-as-empty (done); SEO fallbacks (done). Deviation noted in header: Markdown renders as paragraphs (blog convention) until the blog admin ships a real renderer.

- **Placeholder scan:** every step has complete code; order-page step shows the exact changed lines and names the import.
- **Type consistency:** `Product` fields consumed by card/quick-view/detail all defined in Task 1; `productCta` consumes `status`/`liveUrl` only; `ProductsExplorer` products prop = `Product[]`; jsdom `<dialog>` polyfill lives in the component test only.